

Python 기초

인공지능 개론

Jinu Gong

학습 목표

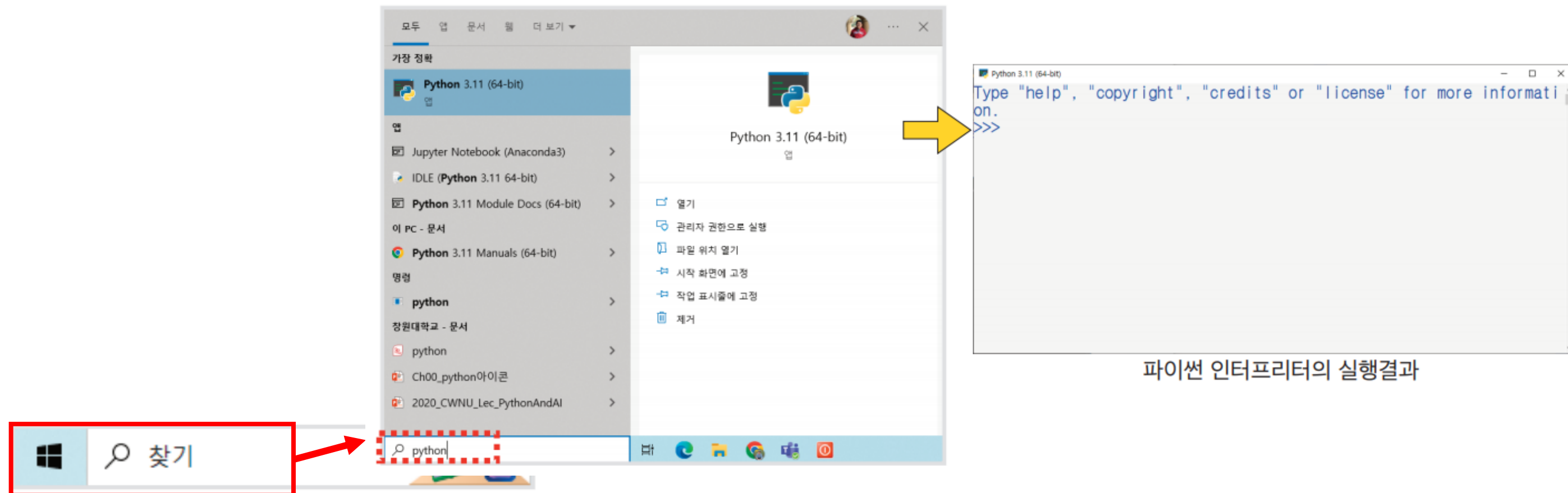
- 파이썬 기초에 대해 이해할 수 있다
- 변수 및 변수를 사용하는 이유에 대해 이해하고 이를 적절히 사용할 수 있다
- 자료형에 대해 알고 데이터에 따른 자료형을 구분할 수 있다
- 더하기, 빼기, 곱하기, 나누기 등의 산술 연산자에 대하여 학습해 본다
- 할당 연산자와 복합 연산자를 활용할 수 있다
- 주석의 개념을 이해해 본다

파이썬 기초

파이썬

•파이썬 인터프리터 사용

- 시작 버튼을 눌러 “python”을 검색 후 Python 3.11을 눌러서 실행
- 파이썬 인터프리터는 처음 자신을 소개한다. 화면 첫 줄에는 자신의 **버전version** 등의 정보를 보여주고 있다. 그리고 다음 줄에는 더 많은 정보를 원할 경우 입력할 수 있는 명령들을 보여 줌



파이썬

- 파이썬 인터프리터 사용

- 사용자의 입력을 받을 수 있는 **프롬프트** prompt에(>>> 표시) 파이썬 명령어 입력
- 프롬프트에 `print('Hello Python!!')` 을 입력 후 엔터키
- 프롬프트 아래에 Hello Python!!이 출력
 - 'Hello Python!!'와 같은 문자의 모음을 컴퓨터 프로그래밍에서는 **문자열** string이라고 함
- `print()`는 파이썬의 **내장함수** built-in function로 괄호 안의 값을 화면에 출력하는 역할
- 인터프리터를 통해 간단한 계산 또한 가능

```
>>> print('Hello Python!!')
Hello Python!!
```

```
>>> 5 + 6
11
>>> 반지름 = 4
>>> 면적 = 3.14 * 반지름 * 반지름
>>> print(면적)
50.24
```


파이썬

- 파이썬 인터프리터 사용

- 파이썬 인터프리터로 코딩을 하는 것은 간단하기는 하지만 잘못 입력한 경우 수정하기가 힘들며, 한 번 일을 시키고 나면, 이 일을 다음에 다시 시키기 어려움
- 이런 문제를 피하는 방법은 파이썬에 입력할 명령어들을 모아 하나의 **소스 코드** `source code`로 저장해 두는 것으로, 소스 코드의 작성과 관리를 돕는 도구를 **통합 개발 도구** `integrated development environment` 혹은 **IDE**라 부름
- 파이썬을 설치하면 간단한 IDE가 제공되는데, 이 도구의 이름이 IDLE (Integrated Development and Learning Environment), "통합적 개발과 학습 환경"이라는 뜻을 가지고 있음

파이썬

- 대화식 모드와 스크립트 모드

- 대화식 모드**는 파이썬 번역기를 실행한 후 >>>라는 프롬프트에서 커서가 깜박일 때 파이썬 명령을 입력하고 엔터키를 치면 번역기가 한 줄 또는 그 이상의 코드를 실행
 - 스크립트 모드**는 새로운 파일을 하나 생성하고 파이썬 명령을 입력하고 나서 이 명령으로 이루어진 코드를 저장해야 하며, 이 때 파일 이름이 file_name.py라고 하면 IDLE에서 " Run " 이라는 메뉴를 선택해서 이 코드를 실행시킬 수 있음

	대화식 모드	스크립트 모드(file_name.py 예시)
예시	<pre>>>> print('당신의 이름은 :') 당신의 이름은 : >>> name = '홍길동' >>> print(name) 홍길동</pre>	<pre>print('당신의 이름은 :') name = '홍길동' print(name)</pre>
실행	파이썬 명령어를 입력하고 엔터키를 입력하면 번역기가 코드를 실행함.	1. 파일을 만들고 저장하여 IDLE에서 "Run" 메뉴를 선택함. 2. python file_name.py 명령을 내림.
장단점	간단한 코드와 문장을 테스트하기에 편리함. 작성한 명령어를 저장할 수 없음.	비교적 긴 파이썬 명령어를 파일로 저장하고 필요할 경우 불러내어 실행함. 짧은 코드를 테스트하기 위해서 번거로운 파일 생성 작업을 해야 함.

The screenshot shows the IDLE Shell 3.11.2 window. The title bar says "IDLE Shell 3.11.2". The menu bar includes "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main text area displays:


```
Python 3.11.2 (tags/v3.11.2:878ead1, Feb 7 2023, 16:38:35) [MSC v.1934 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

 The status bar at the bottom right shows "Ln: 3 Col: 0".

- 이와 같은 파이썬 대화창을 **파이썬 셸** python shell 혹은 **파이썬 대화식 셸**python interactive shell 이라고 함

변수

변수

- **변수**는 컴퓨터의 **메모리**memory 공간에 이름을 붙이는 것으로 우리는 여기에 정수, 실수, 문자열 등의 자료값을 저장 가능
- 저장된 것을 꺼내려면 변수의 이름을 적어주면 됨
- 저장된 값은 수시로 바뀔 수 있고, 그래서 변수(변하는 수)라 명명
- 이 때 사용된 '수'는 수치값이라는 의미가 아닌 **데이터**data로 이해하는 것이 더 정확

변수

- 예를 들어 자신의 몸무게를 weight라는 이름의 변수에 저장하는 경우

```
>>> weight = 78.2
```

- 파이썬 내부에 weight라는 이름의 변수가 생성되고 여기에 78.2가 저장됨
- 위의 코드에 있는 '=' 기호는 '같다'는 등호의 의미가 아니라 '**오른쪽의 값을 왼쪽의 weight라는 이름의 변수에 저장하라**'는 의미
 - 이 연산자 = 를 **할당연산자** 혹은 **대입연산자**라고 함
- 변수는 값을 저장하는 기능이 있으므로 다음과 같이 변수의 값을 출력 가능

```
>>> weight  
78.2
```

변수

- 파이썬의 대화창 콘솔에서는 변수의 이름만 써주어도 변수의 값이 출력되지만 다음과 같은 스크립트 파일(xxx.py 파일을 말함)에서는 반드시 `print(weight)` 함수를 사용하여야 변수의 값을 출력할 수 있음

```
weight = 78.2  
print(weight)
```

```
78.2
```

변수

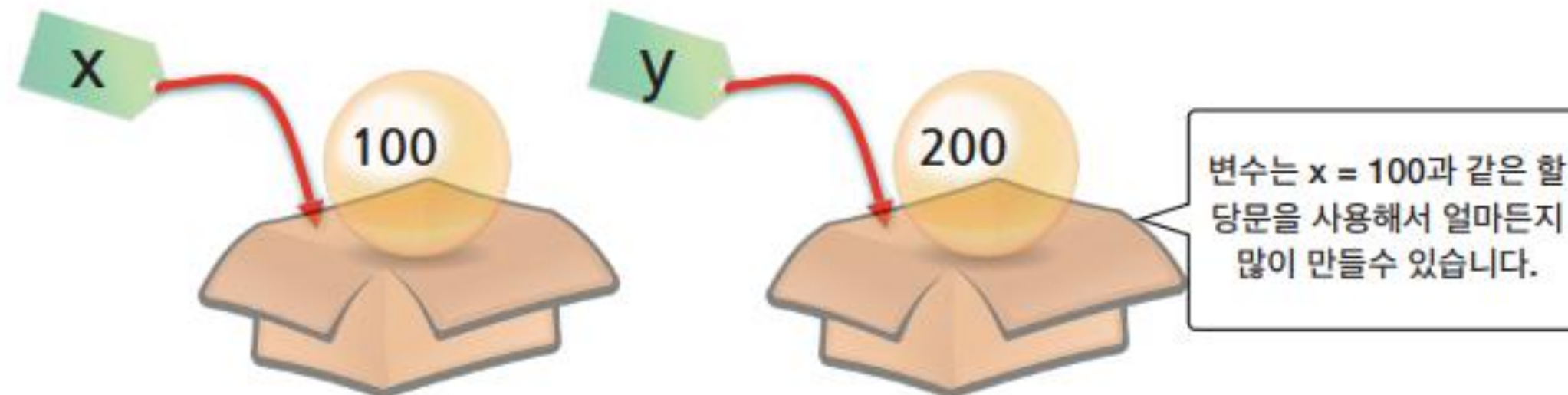
- 내용을 언제든지 바꿀 수 있는 변수
 - 체중이 78.2kg이던 사람이 다이어트를 통하여 76.9kg으로 체중을 줄였다고 가정하면, 이 경우 다음과 같이 체중을 나타내는 변수의 값을 76.9로 바꿀 수 있음

```
>>> weight = 78.2    # weight 변수의 최초값
>>> weight = 76.9    # 변수가 가지고 있는 값은 변경할 수 있다
>>> weight
76.9
```

변수

- 내용을 언제든지 바꿀 수 있는 변수
 - 또한 변수는 필요에 따라서 얼마든지 많이 만들 수 있음
 - 2개의 변수 x와 y를 생성하고 여기에 100과 200을 저장하고, x, y를 다음과 같이 한 행에 쉼표를 입력하면 (100, 200) 형태의 묶음으로 출력

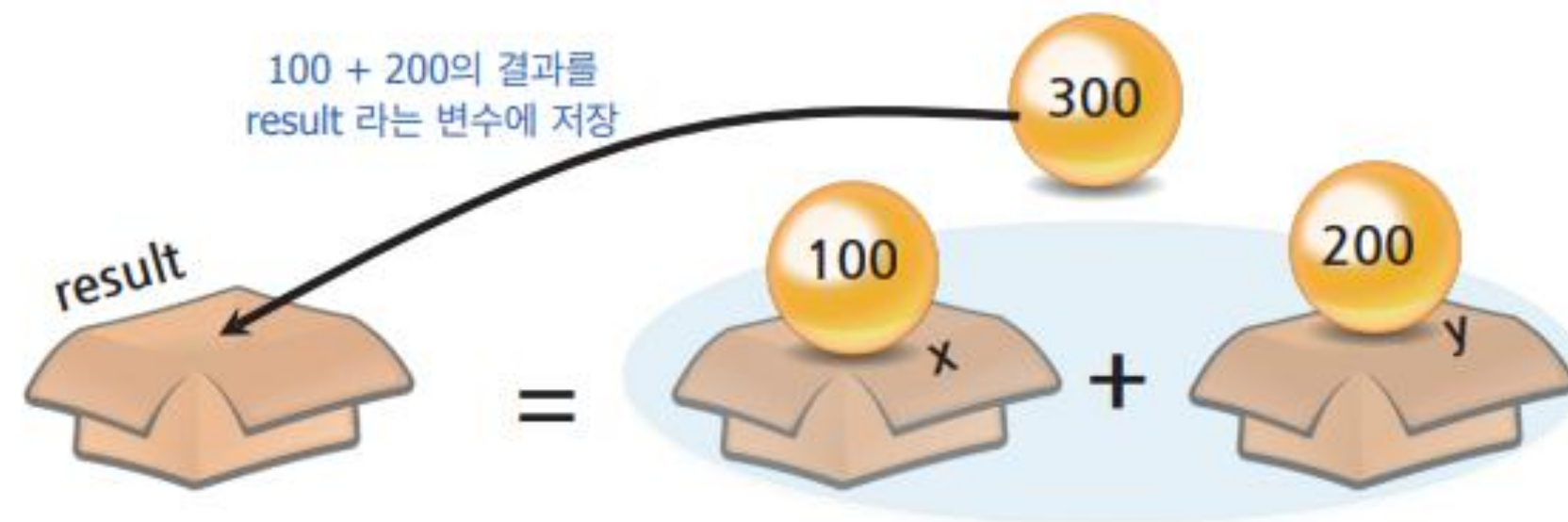
```
>>> x = 100  
>>> y = 200  
>>> x, y  
(100, 200)
```



변수

- 내용을 언제든지 바꿀 수 있는 변수
 - $x, y = 100, 200$ 과 같이 **한 줄에 여러 개의 변수를 선언하고 이 변수에 값을 동시에 할당할** 수도 있는데, 이를 **동시 할당문**이라 함

```
>>> x, y = 100, 200    # x에 100, y에 200을 각각 할당하는 동시 할당문
>>> result = x + y
>>> result
300
```



변수 이름 짓기

- 변수 이름 짓기에 대한 규칙
 - 변수의 이름은 식별자(identifier)의 일종
 - "홍길동", "김철수" 등의 이름이 사람을 구별하듯이 식별자는 변수들을 구별하는 역할

- 식별자는 문자와 숫자, 밑줄 문자(_)로 이루어지며 밑줄 문자 이외의 특수 문자를 사용할 수 없다.
- 식별자의 첫 글자는 숫자로 시작할 수 없다. 또한 중간에 공백을 가질 수 없다.
- 대문자와 소문자는 구별된다. 따라서 변수 `index`와 `INDEX`은 서로 다른 변수이다.
- 파이썬의 예약어(키워드)는 식별자로 사용할 수 없다.

변수 이름 짓기

- 우리는 변수의 이름을 마음대로 지을 수 있지만 파이썬이 내부적으로 사용하는 **예약어는 변수의 이름으로 사용할 수 없음**
 - 예약어를 사용하여 변수를 생성하면 오류 발생
- 파이썬의 예약어

False	class	return	is	finally	None
if	for	lambda	continue	True	def
from	while	nonlocal	and	del	global
not	with	as	elif	try	or
yield	assert	else	import	pass	break
except	in	raise			

변수 이름 짓기

- 변수의 이름을 지을 때는 변수의 역할을 잘 설명하는 이름을 지어야 함
- 예를 들면 체중과 키를 x1, x2라고 하는 것보다 weight와 height라고 하는 편이 이해하기 쉬움



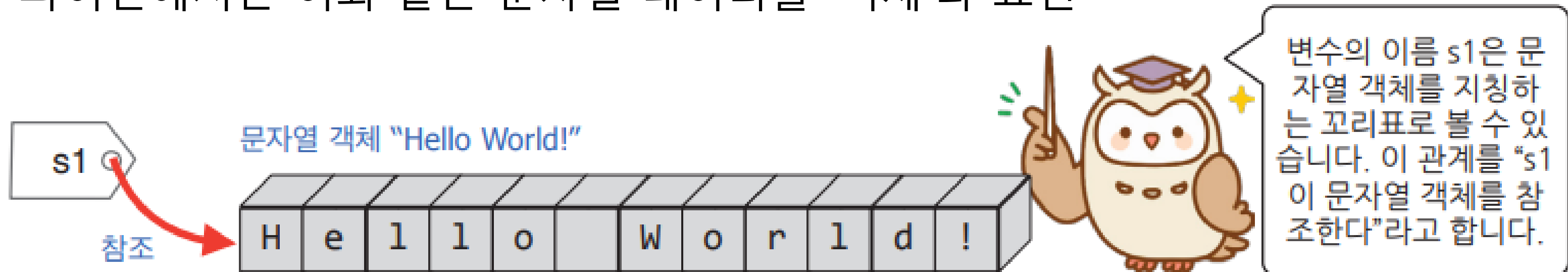
체중과 키를 저장하는 나쁜 변수 이름의 예	체중과 키를 저장하는 좋은 변수 이름의 예
<pre>>>> x1 = 78.2 >>> x2 = 180.0</pre>	<pre>>>> weight = 78.2 >>> height = 180.0</pre>

변수와 문자열

- 변수에는 문자열도 저장할 수 있다
 - **문자열로 이루어진 텍스트 데이터**도 변수에 저장할 수 있음
 - 프로그래밍 세계에서는 텍스트 데이터를 **문자열string**이라고 부름
 - 문자열은 큰따옴표(" ... ")나 작은따옴표(' ... ')를 이용하여 만들 수 있음

```
>>> s1 = 'Hello World!'      # s1 = "Hello World!"과 동일하다
>>> s1
'Hello World!'
```

- 파이썬에서는 이와 같은 문자열 데이터를 '객체'라 표현



변수와 문자열

- 변수에는 문자열도 저장할 수 있다
 - 파이썬은 개발자들이 편리하게 호출하여 사용할 수 있는 미리 정의된 기능을 **함수** `function`라는 형태로 제공
 - `s1` 문자열의 길이는 `len()` 함수 적용

```
>>> len(s1)          # s1 문자열의 길이를 알려준다
12
```

변수와 문자열

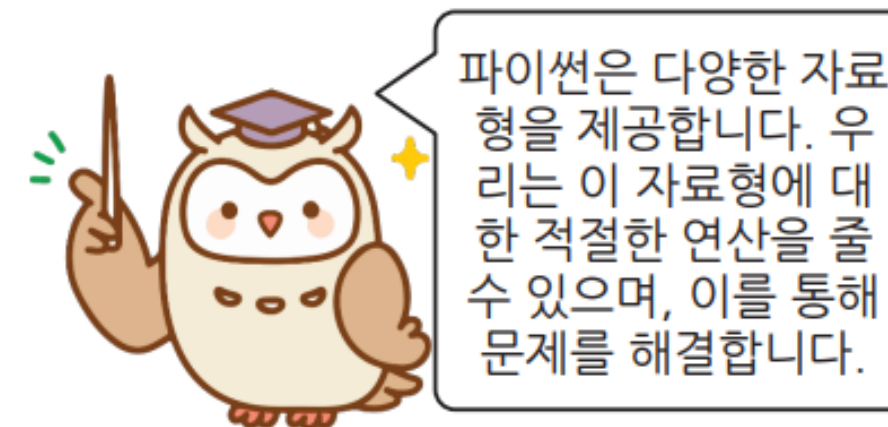
- 변수에는 문자열도 저장할 수 있다
 - 2개의 문자열을 서로 합하려면 다음과 같이 + 연산을 사용

```
>>> s1 = "Hello"
>>> s2 = "World!"
>>> s1 + s2      # s1 문자열과 s2 문자열을 결합한 결과를 반환
'HelloWorld!'
>>> n1 = '100'
>>> n2 = '200'
>>> n1 + n2      # 문자열에 대한 덧셈 연산자는 문자열 이어 붙이기만 수행
'100200'
```

자료형

- 프로그램에서 사용할 수 있는 데이터의 종류를 **자료형** `data type`이라고 함
- 파이썬에서 사용하는 가장 단순한 자료형 4가지
 - 1이나 2와 같은 **정수** `integer`
 - 3.14와 같은 **실수** `floating-point`
 - 'Hello World!'와 같은 **문자열** `string`
 - 참과 거짓을 나타내는 **부울형** `bool`

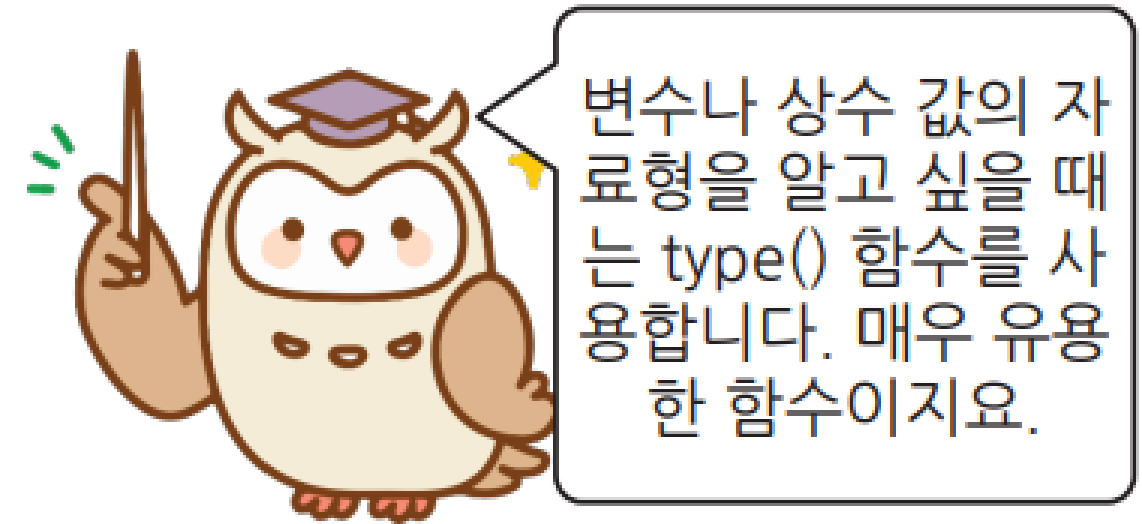
자료형	예
정수형(int)	..., -3, -2, -1, 0, 1, 2, 3, ...
실수형(float)	3.14, 4.28, 0.01, 123.432
문자열(str)	'Hello World', '123', "Hi"
부울형(bool)	True, False
리스트(list)	['ABC', 'DEF'], [3, 4, 5, 6]



자료형

- 파이썬에서 변수의 자료형을 알려면 변수의 이름에 `type()` 함수를 적용

```
>>> weight = 78.2  
>>> type(weight)  
<class 'float'>
```



- 'float' 타입은 파이썬에서 실수를 의미

자료형

- **부울**bool형은 참이나 거짓을 나타냄
 - 부울형은 조건문이나 반복문에서 어떤 조건을 검사할 때 유용

```
>>> b = True
>>> type(b)
<class 'bool'>
```



자료형

- 자료형을 신경써야 하는 이유
 - 파이썬에서 자료형에 따라서 각종 연산의 의미가 달라지기 때문
 - 변수에 정수가 저장되어 있는 경우와 변수에 문자열에 저장된 경우에 덧셈 연산의 의미가 달라짐

```
>>> x = 10
```

x, y는 각각 10, 10을 참조하는 정수형임.

```
>>> y = 10
```

```
>>> x + y
```

두 숫자의 합을 구한다

```
20
```

```
>>> x = 'good'
```

```
>>> y = 'morning!'
```

x, y는 각각 'good', 'morning'을 참조하는 문자열형임.
(자료형이 변경되어 +연산이 하는 일이 달라짐)

```
>>> x + y
```

두 문자열을 연결시킨다

```
'goodmorning!'
```


자료형

- 주의사항

- 파이썬에서 따옴표가 있으면 문자열이고 따옴표가 없으면 숫자

```
>>> '23' + '56'      # 문자열에 대한 + 연산은 문자열을 합치는 기능
'2356'
```

```
>>> 23 + 56          # 정수에 대한 + 연산은 두 정수의 합을 구하는 기능
79
```

연산자

수식

- 할당 연산자assignment operator

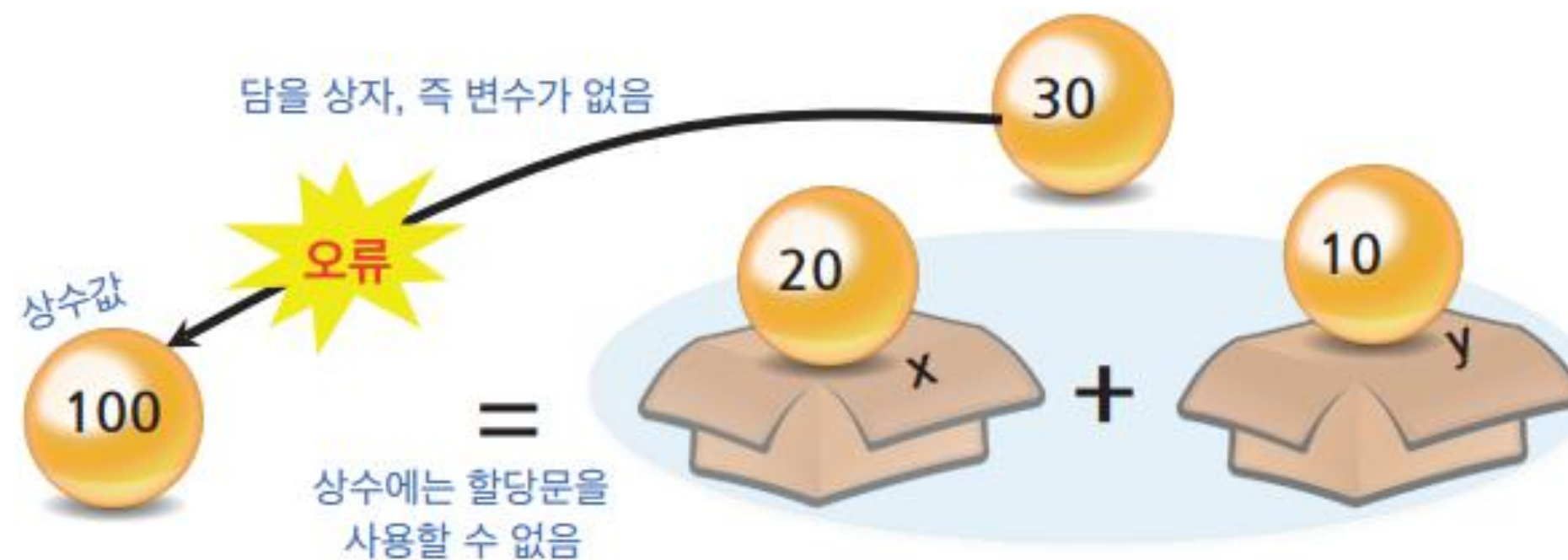
- 변수에 값을 할당할 때 사용하였던 = 기호
- 대입 연산자라고 부르기도 함
- "같다"라는 의미가 아니고, 변수에 값을 저장하는 연산
- = 연산자의 **왼쪽은 반드시 값을 저장할 수 있는 변수**이어야 하고, 오른쪽은 값을 표현하는 숫자나 변수, 혹은 수식expression이 와야 함

```
>>> x = 100 + 200    # x에 100 + 200의 결과를 할당
>>> x
300
```

수식

- 등호의 왼쪽이 변수가 아니면 문제가 발생

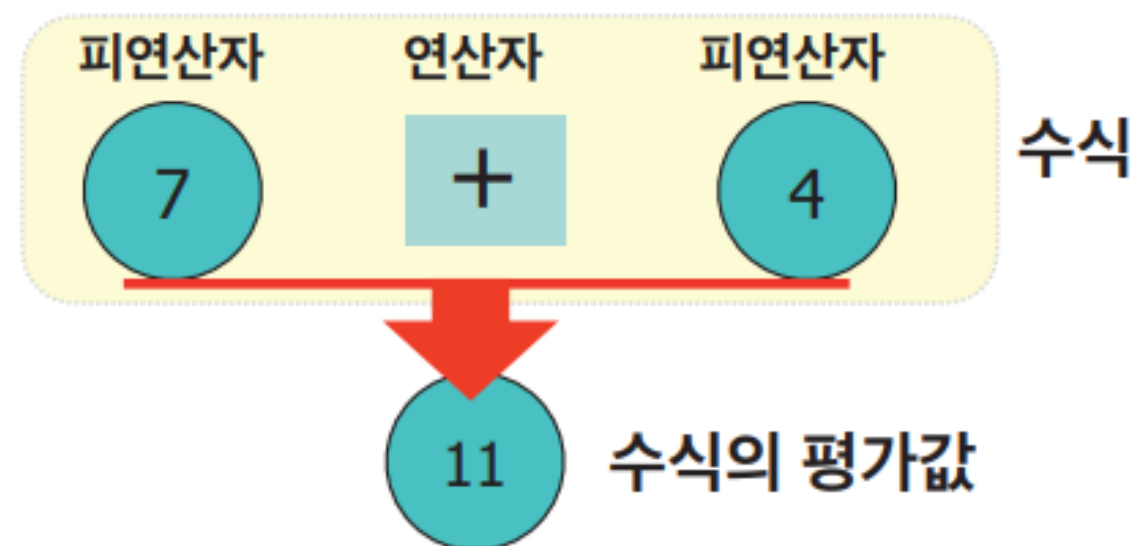
```
>>> x = 20
>>> y = 10
>>> 100 = x + y    # 등호의 왼쪽에는 반드시 변수 이름이 와야 한다
File "<stdin>", line 1
SyntaxError: cannot assign to literal
```



수식과 연산자

- 수식 expression

- 피연산자들과 연산자의 조합
- 연산자 operator는 어떤 연산을 나타내는 기호를 의미
- 피연산자 operand는 연산의 대상이 되는 숫자나 변수를 의미
- 아래의 수식 $7 + 4$ 에서 7과 4는 피연산자이고 덧셈을 의미하는 $+$ 는 연산자
- 연산자와 피연산자 사이에 공백을 삽입하는 것이 파이썬의 코딩 관습에 더 적합한 방식 → 즉 $a*b$ 보다 $a * b$ 로 코딩하는 것이 좋음



수식과 연산자

- 산술 연산자

- 덧셈, 뺄셈, 곱셈, 나눗셈과 나머지 연산을 실행하는 여러가지 연산자

연산자	기호	사용 예	결과값
덧셈	+	$7 + 4$	11
뺄셈	-	$7 - 4$	3
곱셈	*	$7 * 4$	28
지수	**	$7 ** 2$	$7^2 = 49$
나눗셈(정수 나눗셈의 몫)	//	$7 // 4$	1
나눗셈(실수 나눗셈)	/	$7 / 4$	1.75
나머지	%	$7 \% 4$	3

수식과 연산자

- 나눗셈 연산시 주의할 점
 - 파이썬에서 나눗셈 연산 /은 항상 실수로 계산

```
>>> 7 / 4      # 나눗셈은 항상 실수로 계산된다
1.75
>>> 8 / 4      # 나눗셈이기 때문에 정수 2가 아닌 실수 2.0이 출력
2.0
```

- 경우에 따라 나눗셈의 몫을 정수로 계산하고 싶은 경우 //을 사용
- //을 사용하여 나눗셈을 하면 소수점 이하는 없어지고 **정수 부분만 남음**

```
>>> 7 // 4     # //을 사용해서 나누면 정수 몫만 나온다.
1
>>> 8 // 4     # //을 사용해서 나누면 정수 몫만 나온다.
2
>>> 9 // 4     # //을 사용해서 나누면 정수 몫만 나온다.
2
```

수식과 연산자

• 나머지 연산자(%)

- $x \% y$ 는 x 를 y 로 나누어서 남은 나머지를 반환



수식과 연산자

- 몫은 // 연산자가 계산하고, 나머지는 % 연산자로 계산
 - 입력값으로 실수값도 가능

```
p = int(input("분자를 입력하시오: "))    # 실수를 입력받을 때 int() 대신 float() 사용
q = int(input("분모를 입력하시오: "))    # 실수를 입력받을 때 int() 대신 float() 사용
print("나눗셈의 몫 =", p // q)
print("나눗셈의 나머지 =", p % q)
```

```
분자를 입력하시오: 11
분모를 입력하시오: 4
나눗셈의 몫 = 2
나눗셈의 나머지 = 3
```

수식과 연산자

• 거듭제곱^{power}

- 거듭제곱을 계산하려면 ** 연산자를 사용
- 예를 들어서 2^7 을 계산하는 파이썬 수식은 $2 ** 7$
 - 이때 2는 밑(base)이라고 하고 7을 지수(exponent)라고 한다.

```
>>> 2 ** 7          # 2의 7제곱이 계산되며 2*2*2*2*2*2*2와 동일함
128
```

- 수학에서처럼 거듭제곱 연산자는 다른 연산자들보다 높은 우선순위를 가짐

수식과 연산자

- 제곱근 계산
 - 거듭제곱식을 활용하여 계산
 - 피타고라스 정리 예

$$c = \sqrt{a^2 + b^2} = (a^2 + b^2)^{\frac{1}{2}}$$

```
bottom = float(input('직각삼각형의 밑변의 길이를 입력하시오: '))
height = float(input('직각삼각형의 높이를 입력하시오: '))
hypotenuse = (bottom ** 2 + height ** 2) ** 0.5
print('빗변은', hypotenuse, '입니다')
```

```
직각삼각형의 밑변의 길이를 입력하시오: 3
직각삼각형의 높이를 입력하시오: 4
빗변은 5.0 입니다
```

복합 할당 연산자

- 복합 할당 연산자
 - 산술 연산자와 할당 연산자를 결합하여 표현하는 방법
 - 만일 특정한 산술 연산자를 @라고 하면 복합 할당 연산자는 @= 모양

$$a \ @ = \ b \iff a \ = \ a \ @ \ b$$

연산자	사용 방법	의미
<code>+=</code>	<code>i += 10</code>	<code>i = i + 10</code>
<code>-=</code>	<code>i -= 10</code>	<code>i = i - 10</code>
<code>*=</code>	<code>i *= 10</code>	<code>i = i * 10</code>
<code>/=</code>	<code>i /= 10</code>	<code>i = i / 10</code>
<code>//=</code>	<code>i //= 10</code>	<code>i = i // 10</code>
<code>%=</code>	<code>i %= 10</code>	<code>i = i % 10</code>
<code>**=</code>	<code>i **= 10</code>	<code>i = i ** 10</code>

비교 연산자

• 비교 연산자comparison operator

- '크다' 혹은 '작다' 등과 같은 비교 연산
- 그 결과인 True 혹은 False를 반환

연산자	설명	a = 100 b = 200
==	두 피연산자의 값이 같으면 True를 반환	a == b는 False
!=	두 피연산자의 값이 다르면 True를 반환	a != b는 True
>	왼쪽 피연산자가 오른쪽 피연산자보다 클 때 True를 반환	a > b는 False
<	왼쪽 피연산자가 오른쪽 피연산자보다 작을 때 True를 반환	a < b는 True
>=	왼쪽 피연산자가 오른쪽 피연산자보다 크거나 같을 때 True 반환	a >= b는 False
<=	왼쪽 피연산자가 오른쪽 피연산자보다 작거나 같을 때 True 반환	a <= b는 True

논리 연산자

- 논리 연산자logical operator

- 논리 and, or, not 연산을 통해 True나 False 중 하나의 값을 가지는 **부울bool** 값을 반환

연산자	의미
x or y	x나 y중에서 하나라도 참이면 참이 되며, 모두 거짓일 때만 거짓이 된다.
x and y	x와 y중 거짓(False)이 하나라도 있으면 거짓이 되며 모두 참(True)인 경우에만 참이다.
not x	x가 참이면 거짓, x가 거짓이면 참이 된다.

연산자 순서

- 곱셈과 나눗셈이 덧셈과 뺄셈보다 먼저 수행
- 우선순위로 연산을 하지 않고 다른 순서로 하고 싶은 경우 수학 시간에 사용한 수식처럼 이 경우에는 괄호를 사용

$$x + y * z$$

①

②

$$(x + y) * z$$

①

②

연산자 순서

연산자	설명
()	괄호 연산자로 가장 높은 우선 순위 연산자
**	지수 연산자
~ , + , -	단항 연산자(예: -10, +20)
* , / , % , //	곱셈, 나눗셈, 나머지 연산자
+ , -	덧셈, 뺄셈
>> , <<	비트 이동 연산자
&	비트 AND 연산자
^ ,	비트 XOR 연산자, 비트 OR 연산자
<= , < , > , >=	비교 연산자
== , !=	동등 연산자
= , %= , /= , //= , -= , += , *= , **=	할당 연산자, 복합 할당 연산자
is , is not	아이덴티티 연산자
in , not in	소속 연산자
not , or , and	논리 연산자

높은
우선순위



((a + b) * (c - d)) / (e + f)

랜덤 모듈과 math 모듈

- 파이썬은 개발자들이 손쉽게 프로그램을 작성할 수 있도록 다양한 내장함수를 제공하는데 `print()`, `input()`, `len()`, `sum()`, `min()`, `max()`, `int()`, `str()` 등 70여 가지 이상을 제공하고 있음
- 이 뿐 아니라 표준 라이브러리라는 이름으로 터틀 그래픽, 난수 생성기와 다양한 수학 함수 등의 기능을 제공

랜덤 모듈과 math 모듈

- random 모듈

- 임의의 수를 생성하거나, 리스트나 튜플 내의 요소를 무작위로 섞거나, 선택하는 함수를 포함

```
>>> import random
>>> random.random()          # 0 이상 1 미만의 임의의 실수를 반환
0.19452357419514088
>>> random.random()          # 이 함수는 매번 다른 실수를 반환
0.6947454047320903
>>> random.randint(1, 7)      # 1이상 7이하(7을 포함)의 임의의 정수를 반환
3
>>> random.randrange(7)       # 0이상 7미만(7을 포함하지 않음)의 임의의 정수를 반환
3
>>> random.randrange(1, 7)     # 1이상 7미만(7을 포함하지 않음)의 임의의 정수를 반환
6
>>> random.randrange(0, 10, 2) # 0, 2, 4, 6, 8 중(10은 포함 안함) 하나를 반환함
2
>>> lst = [10, 20, 30, 40, 50] # 여러 개의 값을 가지는 리스트를 생성함
>>> random.shuffle(lst)        # lst 리스트의 순서를 무작위로 섞음
>>> lst
[40, 50, 10, 20, 30]
>>> random.choice(lst)         # lst 리스트의 원소 중 무작위로 하나를 고름
30
```

랜덤 모듈과 math 모듈

- math 모듈

- 원주율 π , 자연상수 e 등의 상수 값과 실수의 절대값 `fabs()`, `ceil()`, `floor()`, `log()`, `sin()`, `cos()` 등의 다양한 수학 함수를 포함

```
>>> import math
>>> math.pow(3, 3)           # 3의 3 제곱
27.0
>>> math.fabs(-99)          # -99의 실수 절대값
99.0
>>> math.log(2.71828)
0.9999999327347282
>>> math.log(100, 10)       # 로그 10을 밑으로 하는 100 값
2
>>> math.pi                # 원주율 pi
3.141592653589793
>>> math.sin(math.pi / 2.0) # sin() 함수의 인자로 PI/2.0를 넣어보자
1.0
```

감사합니다

Jinu Gong
